# Multi-Table LEFT JOIN

How Each JOIN Works on the Previous Result

---

## The Query We Are Explaining

This guide explains exactly how the following 6-table LEFT JOIN query works, step by step. The key concept is that each new JOIN operates on the result of all previous JOINs combined, not on the original table.

```
SELECT first_name, last_name, salary,
job_title, department_name, city,
country_name, region_name
FROM hr.employees e
LEFT JOIN hr.departments d ON e.department_id = d.department_id
LEFT JOIN hr.jobs j ON e.job_id = j.job_id
LEFT JOIN hr.locations l ON d.location_id = l.location_id
LEFT JOIN hr.countries c ON l.country_id = c.country_id
LEFT JOIN hr.regions r ON c.region_id = c.region_id;
```

## Complete JOIN Chain Overview

Look at the full picture below. Six tables are joined one after another. Notice how each JOIN uses columns from the accumulated result (not the original tables). The highlighted step (Step 4) shows the most important detail: d.location_id comes from the accumulated result, not directly from the departments table.

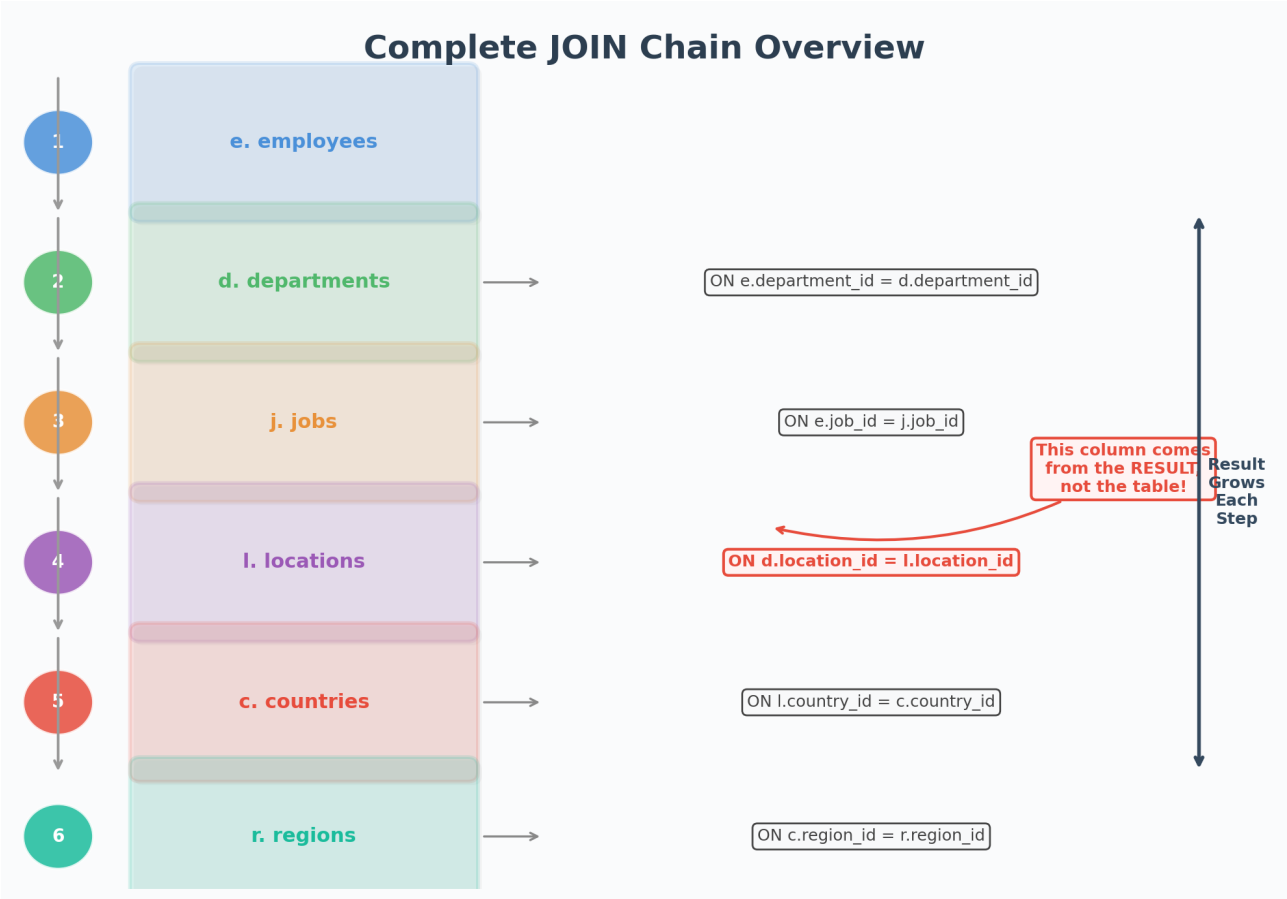


Figure 1: All 6 JOIN steps and their ON conditions

# Step-by-Step Breakdown

## Step 1: The Starting Point

Every multi-table query starts with one base table. In our case, it is hr.employees (alias: e). This table contains the core data we want: employee names, salaries, and reference keys (job_id, department_id). Think of this as our foundation. Everything else will be added on top of it.


Figure 2: We begin with the employees table as our base

## Step 2: Add Department Names

Now we add department information. We use LEFT JOIN to connect employees with departments. The ON clause matches e.department_id = d.department_id. The result now contains ALL employee rows (even those without a department), plus the department_name column for matched rows. If an employee has no department, department_name will be NULL.

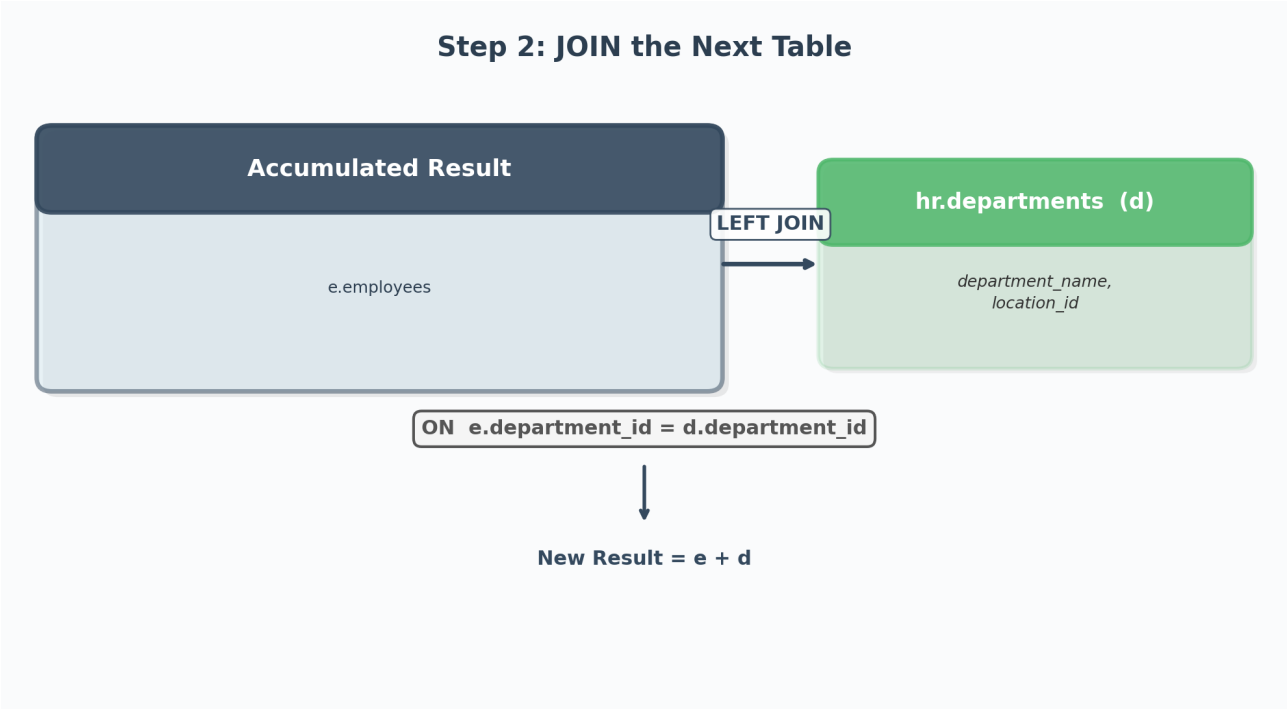


Figure 3: Employees + Departments = Result with department names

# Step 3: Add Job Titles

Next, we add job titles. Importantly, this JOIN operates on the result from Step 2 (employees + departments), NOT just the original employees table. The ON clause uses e.job_id = j.job_id. The "e" alias still works because the employee columns (including job_id) are carried forward in the accumulated result. The result now has three sources of data merged together.

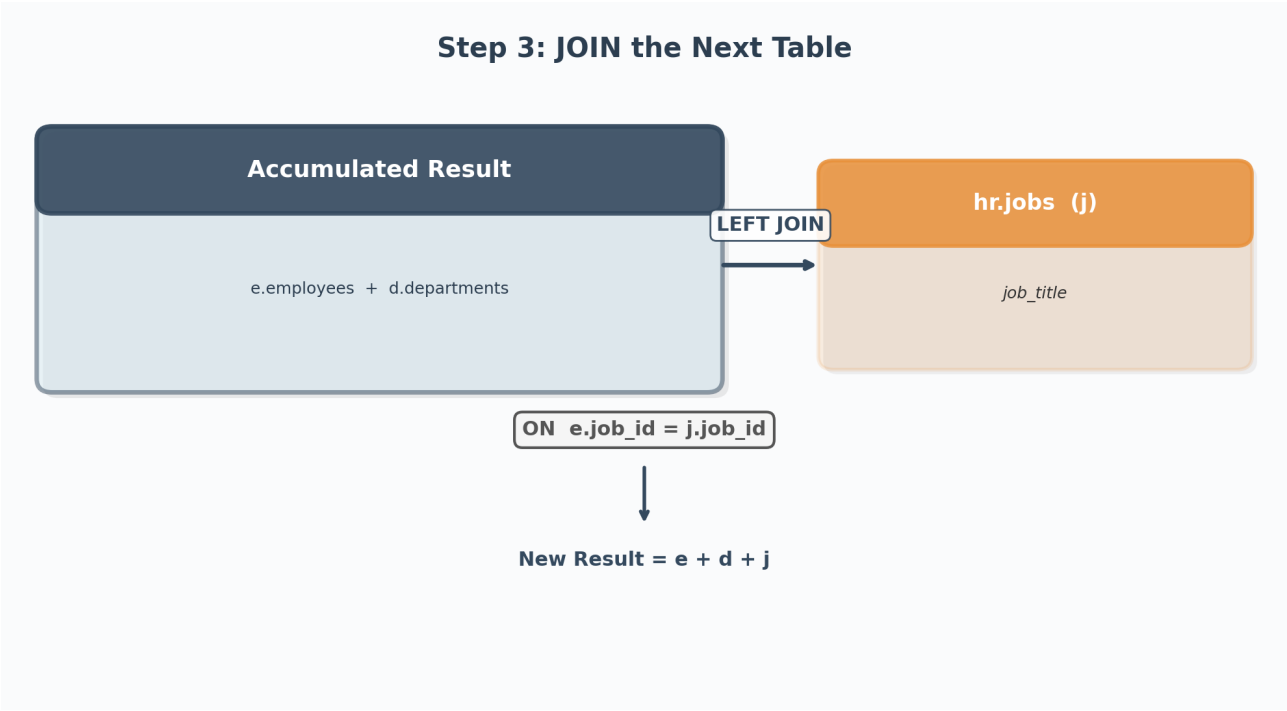


Figure 4: Previous result + Jobs = Result with job titles

# Step 4: Add Locations (Important!)

This is the most important step to understand. Look at the ON clause carefully:

**ON d.location_id = l.location_id**

Notice it uses d.location_id. At this point, the SQL engine is NOT looking at the original departments table. Instead, it is looking at the accumulated result from Steps 1-3 (which already contains columns from employees, departments, and jobs). The "d" alias refers to the departments data that was already merged into the result back in Step 2. This is the key concept: each JOIN builds on the combined output of all previous JOINs. The "d.location_id" value is read from the intermediate result set, not from the departments table directly.

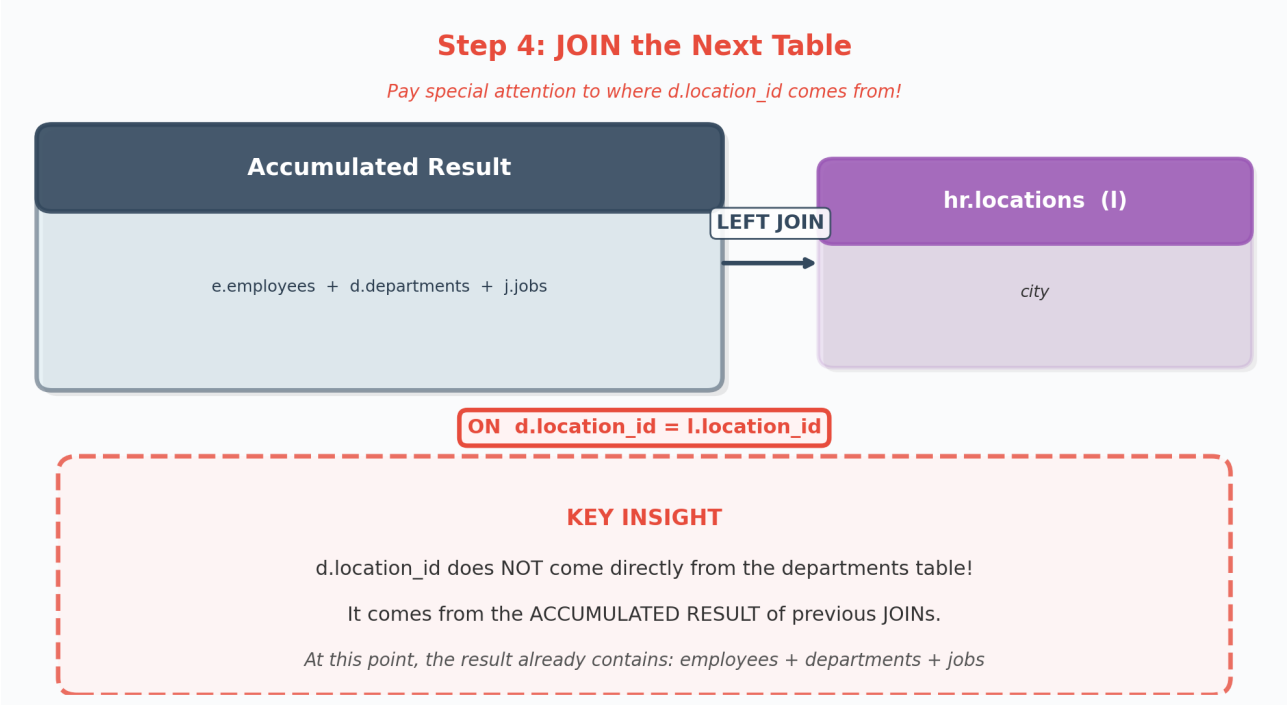


Figure 5: The d.location_id column comes from the accumulated result, NOT directly from the departments table!

# Step 5: Add Countries

Now we add country names. The ON clause is l.country_id = c.country_id. Again, "l" refers to the locations data that was merged in Step 4. The accumulated result now contains data from four tables: employees, departments, jobs, and locations. Each LEFT JOIN preserves all rows from the left side (the accumulated result) and adds matching data from the new table on the right.

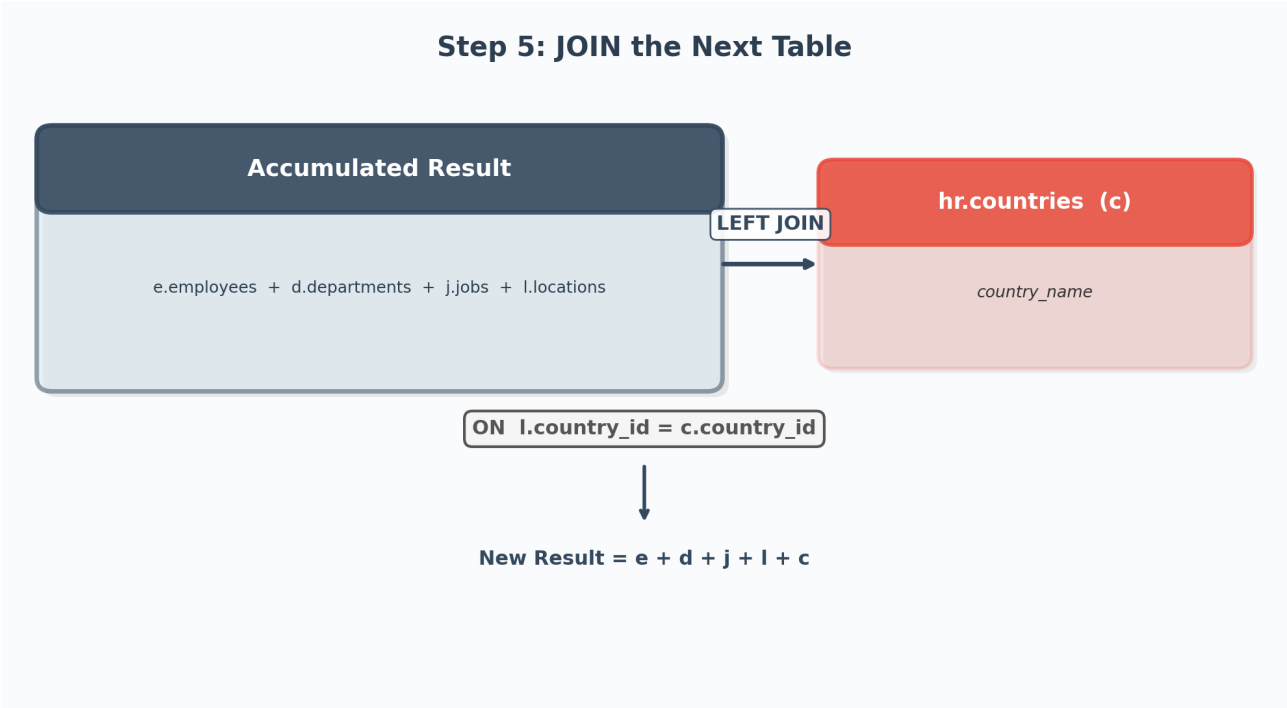


Figure 6: Previous result + Countries = Result with country names

# Step 6: Add Regions

The final step adds region names. The ON clause is c.region_id = r.region_id. After this JOIN, the complete result contains columns from all six tables. Every employee row is preserved (because we used LEFT JOIN at every step), and we now have the full chain: employee details, their department, job title, office location, country, and region.

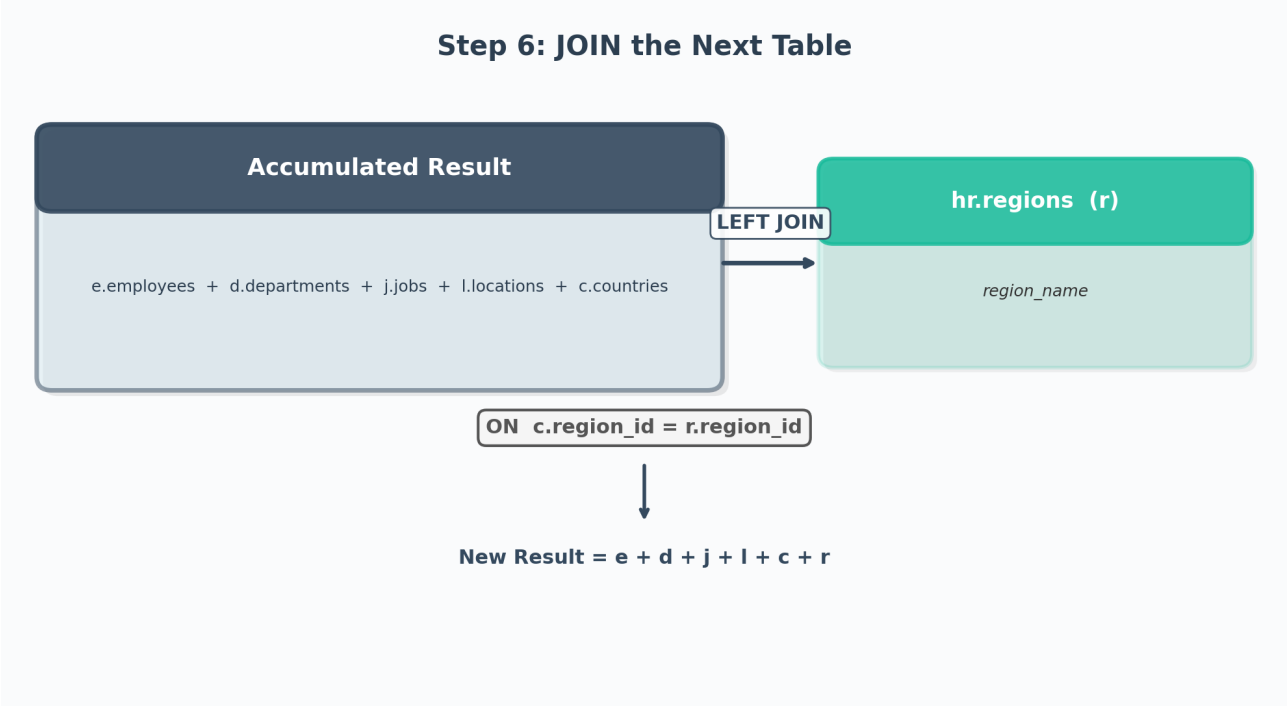


Figure 7: Previous result + Regions = Final complete result

# Summary: The Golden Rule of Multi-Table JOINs

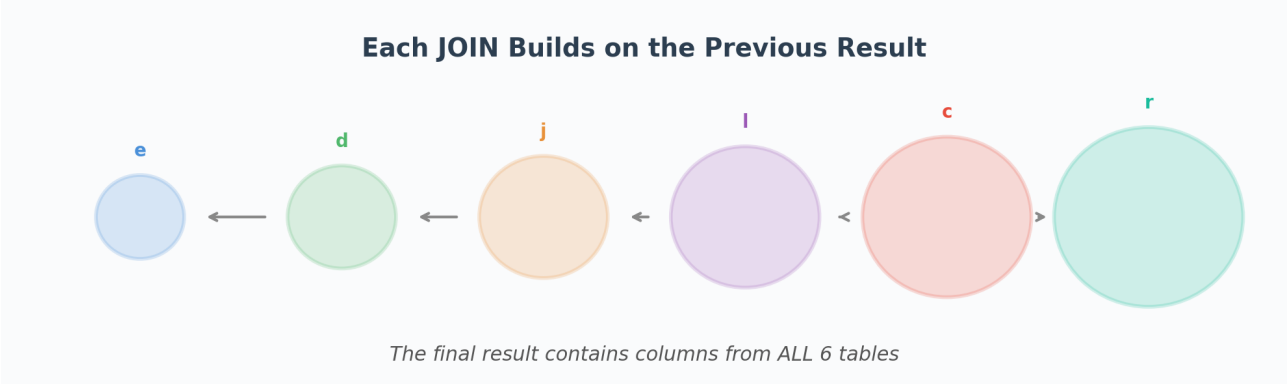


Figure 8: The result grows with each JOIN step

When you write a query with multiple JOINs, remember this fundamental rule: each JOIN operates on the combined result of all previous JOINs. The SQL engine processes JOINs in order, from left to right (top to bottom in your query). After each JOIN, the result becomes the new "left side" for the next JOIN.

This means that column aliases like "d.location_id" or "e.job_id" in later ON clauses do NOT reference the original tables. They reference the columns that already exist in the intermediate result set. The departments table was joined back in Step 2, so by Step 4, its columns (including location_id) are already part of the result. The SQL engine simply reads them from there.

## Key Takeaways

1. LEFT JOIN preserves all rows from the left side (accumulated result). If no match is found in the new table, NULL values are added for that table's columns.

2. Each JOIN builds on the previous result. It does NOT go back to the original tables. The "d" in step 4 refers to data already merged in step 2.

3. The order of JOINs matters. Each ON clause can only reference tables that have already been included in the accumulated result (either the base table or previously joined tables).

4. Column aliases carry forward. Once a table is joined and gets an alias (like "d" for departments), that alias is available for all subsequent JOINs.